STANDARD OPERATING
PROCEDURE

# Mobile Application Penetration Testing

Android + iOS

| | |
|---|---|
| **Document type** | Standard Operating Procedure (SOP) |
| **Coverage** | Assessment workflow, tools, libraries, risk triage, reporting and learning resources |
| **Source basis** | Mobile Pentest Master Reference supplied by the user |
| **Version** | 1.0 |
| **Prepared** | 25 October 2026 |

**Authorized testing only.** Perform all activities only on applications and environments you own or for which you have explicit written authorization.

# Contents

1. Purpose and Scope

2. Pre-Assessment Checklist

3. Recommended Assessment Workflow

4. Cross-Platform Tool Reference

5. Android Procedure and Tools

6. iOS Procedure and Tools

7. Utilities and Environment Notes

8. Library and Framework Security Review

9. Exploitability and Risk Interpretation

10. Reporting and Closure Checklist

11. Free Learning Resources

12. Reference Links and Limitations

Document control note: This SOP is organized from the supplied master reference. Links, tool behavior, and vulnerability applicability should be re-verified before use.

# 1. Purpose and Scope

This Standard Operating Procedure (SOP) organizes the supplied mobile penetration-testing master reference into a repeatable workflow for Android and iOS application security assessments. It covers tool selection, static and dynamic analysis, supporting utilities, library/framework security considerations, and learning/reference resources.

Use this SOP only for applications and environments you own or for which you have explicit written authorization. Confirm the approved application, build, environment, test accounts, and scope before testing.

# 2. Pre-Assessment Checklist

[ ] Confirm written authorization and approved assessment scope.

[ ] Identify platform: Android, iOS, or both; record application name, version/build, and test environment.

[ ] Confirm device/emulator availability and whether root or jailbreak is permitted/required for planned tests.

[ ] Select proxy, static-analysis, and runtime-analysis tools appropriate to the platform and engagement.

[ ] Record tool versions and verify the relevant tool documentation, as command syntax and flags may change.

[ ] Plan how evidence and findings will be recorded in accordance with the engagement's reporting process.

# 3. Recommended Assessment Workflow

| | |
|---|---|
| **Scope and setup** | Validate authorization, targets, app build, test accounts, device/emulator configuration, and permitted test techniques. |
| **Initial automated review** | Use MobSF for automated static and dynamic analysis where suitable. Treat results as leads that require manual validation. |
| **Static analysis** | Inspect package contents, source/decompiled code, manifests or binary metadata, endpoints, embedded configuration, and dependency/library usage using the platform-appropriate tools listed below. |
| **Traffic analysis** | Configure Burp Suite, OWASP ZAP, or mitmproxy for authorized traffic interception. Verify device proxy and certificate configuration, and document traffic that cannot be observed. |
| **Runtime analysis** | Use Frida, Objection, RMS, or platform-specific utilities for runtime observation and validation where permitted. Some checks require root/jailbreak or a compatible emulator/device. |
| **Security control validation** | Assess relevant controls and implementation risks, including certificate validation/pinning, root/jailbreak detection, exposed components, logging, local data storage, and networking trust configuration. |
| **Finding validation** | Reproduce candidate issues, distinguish confirmed behavior from tool output or assumptions, and record evidence, impact, affected component, and conditions required. |
| **Reporting and closure** | Summarize confirmed findings, practical impact, remediation guidance, limitations, and testing constraints. Re-test fixes only within the approved scope. |

# 4. Cross-Platform Tool Reference

**MobSF (Mobile Security Framework)**

Automated static and dynamic analysis for mobile packages.

https://github.com/MobSF/Mobile-Security-Framework-MobSF

**Frida**

Dynamic instrumentation and runtime hooking.

https://frida.re

**Objection**

Runtime exploration toolkit powered by Frida.

https://github.com/sensepost/objection

**RMS (Runtime Mobile Security)**

Web UI for runtime manipulation on Android and iOS.

https://github.com/mobile-hacking-village/RMS-Runtime-Mobile-Security

**Burp Suite**

Traffic interception and web security testing.

https://portswigger.net/burp

**OWASP ZAP**

Open-source proxy for traffic interception.

https://www.zaproxy.org

**mitmproxy**

Scriptable CLI/web proxy for man-in-the-middle testing.

https://mitmproxy.org

**Ghidra**

Multi-platform disassembler/debugger.

https://ghidra-sre.org

**IDA Pro / IDA Free**

Disassembler/debugger for binary analysis.

https://hex-rays.com/ida-free

**Radare2**

Reverse-engineering framework.

https://rada.re

# 5. Android Procedure and Tools

Use the following tools according to the assessment objective. Tool availability and prerequisites vary by device, operating system, and version.

## 5.1 Static analysis and reverse engineering

**Jadx (GUI/CLI)**

Decompile DEX to Java-like source for review.

https://github.com/skylot/jadx

**Apktool**

Disassemble and reassemble APK resources and application package contents.

https://apktool.org

**dex2jar**

Convert DEX files to JAR for analysis.

https://github.com/pxb1988/dex2jar

**Androguard**

Android bytecode analysis suite.

https://github.com/androguard/androguard

**JD-GUI**

Java decompiler GUI.

https://java-decompiler.github.io

**Luyten**

Alternative Java decompiler GUI.

https://github.com/deathmarine/Luyten

**apkleaks**

Scan APKs for URIs, endpoints, and possible secrets.

https://github.com/dwisiswant0/apkleaks

## 5.2 Dynamic/runtime analysis

**Drozer**

Test Android attack surface, including exported components.

https://github.com/WithSecureLabs/drozer

**pidcat**

Colored logcat output filtered by package name.

https://github.com/JakeWharton/pidcat

**fridump**

Memory dumping via Frida; use only where authorized and permitted.

https://github.com/Nightbringer21/fridump

### 5.3 Device utilities

**ADB (Android Debug Bridge)**

Command-line communication with Android devices.

https://developer.android.com/tools/adb

**scrcpy**

Display and control an Android device over USB/TCP.

https://github.com/Genymobile/scrcpy

## 6. iOS Procedure and Tools

Many iOS dynamic-analysis and binary-dumping tools require a jailbroken device. Confirm device constraints and approval before attempting such techniques.

### 6.1 Decryption and binary dumping

**bagbak**

App Store binary decryption utility (as described in the reference).

https://github.com/ChiChou/bagbak

**Clutch**

Binary decryption for jailbroken devices.

https://github.com/KJCracks/Clutch

**Dumpdecrypted**

Dumps decrypted Mach-O from encrypted apps.

https://github.com/stefanesser/dumpdecrypted

**frida-ios-dump**

Frida-based alternative for app dumping.

https://github.com/AloneMonkey/frida-ios-dump

### 6.2 Binary and runtime analysis

**class-dump**

Extract Objective-C runtime information from Mach-O.

https://github.com/nygard/class-dump

**otool**

Inspect Mach-O headers and linked libraries; included with Xcode command-line tools.

**Cycript (or Frida fork)**

Runtime exploration using Objective-C/JavaScript.

http://www.cycript.org

**bfinject**

Dylib injection utility; use only for authorized testing.

https://github.com/BishopFox/bfinject

**idb**

iOS penetration-testing/research helper.

https://github.com/dmayer/idb

### 6.3 Security-bypass references

**SSL Kill Switch 2**

Disables SSL certificate validation/pinning in supported contexts.

https://github.com/nabla-c0d3/ssl-kill-switch2

**Xcon**

Jailbreak-detection bypass tool referenced for Cydia.

**tsProtector**

Jailbreak-detection bypass tool referenced for Cydia.

**keychaindumper**

Dumps accessible Keychain items on jailbroken devices.

https://github.com/ptoomey3/Keychain-Dumper

## 7. Utilities and Environment Notes

**otool**

macOS command-line utility for Mach-O inspection; ships with Xcode command-line tools.

**balenaEtcher**

OS image flasher for SD/USB media; may be used to flash NetHunter or custom Android ROMs. It is not a penetration-testing tool.

https://etcher.balena.io

**Baleno**

Network-management GUI for serial, SSH, TFTP, IP scanning, SNMP, and vulnerability scanning; unrelated to mobile app penetration testing.

https://aur.archlinux.org/packages/baleno-git

The source reference specifically cautions users to obtain balenaEtcher from its official site and links to a warning about fake software claiming to be balenaEtcher.

## 8. Library and Framework Security Review

### 8.1 Android

**Flutter / Dart SDK**

Review SDK and plugin versions and relevant advisories. The supplied reference lists CVE-2022-3095, CVE-2026-27704, and plugin issues CVE-2024-54461/54462; verify applicability and current status before reporting.

https://vuldb.com/cve/CVE-2022-3095

**PairIP / libpairipcore.so**

App-protection and anti-analysis mechanisms may validate install source/signature and impede instrumentation. Document observed protections and test only within the approved scope.

https://modules.lsposed.org/module/io.github.ahmedmani.pairipfix/

**Third-party logging (Timber, SLF4J, Firebase)**

Inspect logs and analytics for sensitive data exposure, local buffering, or unintended transmission.

https://www.computer.org/csdl/proceedings-article/saner/2026/858200a468/2hNeGHStqZW

### 8.2 iOS

**Alamofire**

Review certificate validation and custom trust policies; insecure configuration can enable man-in-the-middle risk.

https://db.fluidattacks.com/fix/swift/147/

**SDWebImage**

Review image-loading dependencies and underlying codecs; keep dependencies patched and consider malformed-image/resource-exhaustion risks.

https://github.com/xvnpw/sec-docs/blob/main/objectivec/sdwebimage/sdwebimage/2025-02-14-gemini-2.0-pro-exp/attack-surface.md

**CocoaLumberjack**

Review production log content and file-path construction for sensitive data exposure or unsafe path handling.

https://github.com/xvnpw/sec-docs

**Realm / Core Data**

Check whether sensitive local database content is appropriately protected; risk depends on application configuration and device access.

## 9. Exploitability and Risk Interpretation

Use these notes as triage prompts, not as proof that a particular application is vulnerable. Confirm the actual version, configuration, reachable attack path, prerequisites, and impact.

- PairIP: the source reference reports public bypasses, while noting that protections can increase the effort required for casual modification.
- Flutter/Dart: the source reference lists core SDK and plugin CVEs; check affected versions and advisories before determining applicability.
- Logging: sensitive data in logs or analytics can create privacy and data-exposure risk even when no code execution is possible.
- Alamofire: risk is configuration-dependent; inspect trust evaluation and pinning implementation.
- SDWebImage: assess dependency/codec patch status and the application's exposure to untrusted image content.
- iOS libraries generally: risks may arise from implementation choices such as missing certificate pinning or unencrypted local databases, rather than from a library CVE itself.

## 10. Reporting and Closure Checklist

[ ] Record application/platform/build, environment, test date, tools and versions, and scope limitations.

[ ] For each finding, record reproducible steps, evidence, affected component, prerequisites, security impact, and remediation guidance.

[ ] Separate confirmed findings from suspected issues, automated tool alerts, and general library risks.

[ ] Record untested areas and blockers, including unavailable devices, root/jailbreak restrictions, or traffic that could not be intercepted.

[ ] Confirm all testing remained within written authorization and approved targets.

[ ] Provide the report through the agreed process and perform remediation retesting only when authorized.

# 11. Free Learning Resources

**8kSec Battlegrounds**

Free interactive Android/iOS challenges; the reference mentions Frida, Objection, and SSL-pinning topics.

https://8ksec.io

**Mobile Hacking Lab**

Guides and free course content, including YouTube resources.

https://www.mobilehackinglab.com

**Bugcrowd Mobile Hacking Resource Kit**

Curated guides, videos, courses, and tools.

https://www.bugcrowd.com

**RingSafe Academy**

The reference lists a free-tier mobile application penetration-testing learning path.

https://ringsafe.in

**OWASP MASTG / MASVS**

Testing guide and mobile application security verification standard.

https://mas.owasp.org/MASTG/

# 12. Reference Links and Limitations

Useful links from the supplied master reference:

**Frida CodeShare**

Community scripts and examples.

https://codeshare.frida.re

**OWASP MASTG**

Mobile Application Security Testing Guide.

https://mas.owasp.org/MASTG/

**OWASP MASVS**

Mobile Application Security Verification Standard.

https://mas.owasp.org/MASVS/

**MobSF**

Mobile Security Framework.

https://github.com/MobSF/Mobile-Security-Framework-MobSF

**Objection**

Runtime exploration toolkit.

https://github.com/sensepost/objection

**Jadx**

Android decompiler.

https://github.com/skylot/jadx

**Drozer**

Android attack-surface testing tool.

https://github.com/WithSecureLabs/drozer

Links and vulnerability references were carried over from the supplied file and may change. Verify current tool status, official documentation, and vulnerability applicability before relying on them. This SOP is a structured workflow and tool reference; it does not replace the engagement's formal scope, test plan, evidence-handling requirements, or reporting template.